25.9.2 Snarkifying Mithiril

Wuyunsiqin Bingsheng Zhang
Zhejiang University, CHN
22521299@zju.edu.cn bingsheng@zju.edu.cn

September 9, 2025

1 Comparison with Jia Liu's work

Table 1: Comparison between our implementation and Jia Liu's work

Aspect	**Jia Liu's work**	**Our Implementation**
Elliptic Curve	Jubjub curve	BN254 curve
Signature Scheme	Modified into a **deterministic scheme**	Original **BLS signature** used in Mithril
MSM Operation	**MSM2 optimisation** (circuit layout modification to handle two multiplications at once, reducing rows per verification)	variable_base_msm_custom function from halo2-lib
Hardware Environment	AWS instance r7i.12xlarge with 48 vCPU and 384GB memory	Local server with 48-core Intel Xeon Silver 4214 CPU and 128 GB memory

Table 2: Certificate circuit with **msm2 optimisation**

Degree	**Advice rows**	**Quorum_max**	**Proving time**	**Peak memory**	**Verifying time**
11	1752	1	365.744ms	–	8.854ms
12	3240	2	549.739ms	–	7.696ms
13	7704	5	846.477ms	–	8.879ms
14	15144	10	1.517s	–	9.482ms
15	31512	21	2.947s	–	7.460ms
16	64248	43	6.057s	–	7.782ms
17	129720	87	12.404s	–	8.372ms
18	260664	175	25.135s	14GB	8.162ms
19	522552	351	50.860s	22GB	7.034ms
20	1046328	703	103.608s	37GB	7.063ms
21	2092392	1406	208.313s	69GB	7.582ms
22	4184520	2812	428.707s	136GB	8.300ms

Proving 2272 signatures (degree 22): proving time 415.945s, peak memory 136GB.

Table 3: Certificate circuit **without msm2 optimisation**

Degree	**Advice rows**	**Quorum_max**	**Proving time**	**Peak memory**	**Verifying time**
11	–	–	–	–	–
12	2264	1	434.322ms	–	6.853ms
13	6264	3	700.636ms	–	7.464ms
14	16264	8	1.437s	–	6.578ms
15	32264	16	2.732s	–	7.394ms
16	64264	32	5.397s	–	7.476ms
17	130264	65	10.994s	–	6.939ms
18	260264	130	21.930s	9GB	6.017ms
19	522264	261	44.785s	13GB	7.571ms
20	1046264	523	90.909s	21GB	6.969ms
21	2092264	1046	180.081s	38GB	6.758ms
22	4186264	2093	363.844s	69GB	6.961ms
23	8374264	4187	743.101s	131GB	6.838ms

Proving 2272 signatures (degree 23): proving time 581.347s, peak memory 135GB.

Given a secret key sk and its verification key $\mathsf{vk} = g^{\mathsf{sk}}$. A deterministic signature (σ, π) can be generated as follows:

- Compute $\sigma = H_1(\text{msg})^{\mathsf{sk}}$
- Choose random r, compute $R_1 = H_1(\text{msg})^r, R_2 = g^r$
- Compute $c = H_2(g, H_1(\text{msg}), \mathsf{vk}, \sigma, R_1, R_2)$ and $s = \mathsf{sk} \cdot c + r$
- $\pi = (s, c)$

(σ, π) can be verified as:

- Compute $\tilde{R}_1 = H_1(\text{msg})^s / \sigma^c$ and $\tilde{R}_2 = g^s / \mathsf{vk}^c$
- Compute $\tilde{c} = H_2(g, H_1(\text{msg}), \mathsf{vk}, \sigma, \tilde{R}_1, \tilde{R}_2)$
- Verify if $c = \tilde{c}$

Signature generation is similar to above except that the final signature is

$$\pi = (R_1, R_2, s)$$

Batch verification of a single (σ, π):

- Compute $c = H_2(g, H_1(\text{msg}), \mathsf{vk}, \sigma, R_1, R_2)$
- Compute $a = H_3(c, s)$ and verify MSM:

$$R_1 R_2^a \sigma^c \mathsf{vk}^{ca} H_1(\text{msg})^{-s} g^{-as} \stackrel{?}{=} 1$$

Note: not sure if this saves any computation compared to the direct computation of $\tilde{R}_1, \tilde{R}_2$: $\tilde{R}_1 = H_1(\text{msg})^s \sigma^{-c}$ and $\tilde{R}_2 = g^s \mathsf{vk}^{-c}$.

Batch verification of multiple signatures $\{(\sigma_i, \pi_i)\}_{1 \le i \le n}$ with $\pi_i = (R_{1,i}, R_{2,i}, s_i)$:

- For $1 \le i \le n$, compute $c_i = H_2(g, H_1(\text{msg}), \mathsf{vk}_i, \sigma_i, R_{1,i}, R_{2,i})$
- Compute $\rho = H_4(c_1, \ldots, c_n, s_1, \ldots, s_n)$. For $1 \le i \le n$, compute $a_{i,0} = H_5(\rho, i, 0)$ and $a_{i,1} = H_5(\rho, i, 1)$
- Verify MSM:

$$H_1(\text{msg})^{-\sum_{i=1}^n s_i a_{i,0}} g^{-\sum_{i=1}^n s_i a_{i,1}} \prod_{1 \le i \le n} R_{1,i}^{a_{i,0}} R_{2,i}^{a_{i,1}} \sigma_i^{c_i a_{i,0}} \mathsf{vk}_i^{c_i a_{i,1}} = 1$$

2 Snarkifying Mithril (Practical)

This section implements a Halo2-based zero-knowledge cross-chain protocol instantiated with Mithril. The circuit covers: (i) unweighted BLS multisignature aggregation (MSP.A), (ii) seed-weighted BLS aggregation (MSP.B), (iii) lottery index range & uniqueness checks, (iv) eligibility/threshold checks against stake, and (v) Merkle membership checks for registered signers. The experimental environment matches the previous section.

2.1 Experimental Tasks

- **MSP.A (baseline aggregation).** Implements the straightforward BLS aggregation and in-circuit pairing check $e(\mu, g_2) = e(H_{G_1}(\text{"M"} \| msg), ivk)$. We measure proving time, proof size, and verification time as the number of aggregated signatures grows.
- **MSP.B (seed-weighted aggregation).** Extends aggregation with per-signer short exponents $e_i \leftarrow H_\lambda(i, e_{\boldsymbol{\sigma}})$ and $(\mu, e_{\boldsymbol{\sigma}})$ where $e_{\boldsymbol{\sigma}} \leftarrow H_p(\boldsymbol{\sigma})$. The circuit proves the weighted pairing equation. We observe higher proving cost due to hashing and multi-scalar multiplications (MSMs) with weights.
- **Lottery index check.** Ensures $\mathsf{index}_i \leq m$ for all i and $\mathsf{index}_i \neq \mathsf{index}_j$ for $i \neq j$. The uniqueness constraint induces $O(n^2)$ comparisons; we report scaling behavior.
- **Eligibility/threshold check.** For each signer, enforces $ev_i = \mathsf{MSP.Eval}(\mathsf{topic}, \mathsf{index}_i, \sigma_i)$ and $ev_i \leq \phi(\mathsf{stake}_i)$. The circuit cost grows roughly linearly in the number of checked signers.

2.2 Benchmark Tables

Table 4: MSP.A (unweighted aggregation) results

aggregated signatures	**2**	**200**	**2000**	**4000**	**6000**	**8000**
Proving time (s)	19.17	29.51	72.89	123.67	181.38	226.84
Proof size (bytes)	3072	4128	11424	18944	26944	34944
Verification time (ms)	20.93	9.09	19.95	16.03	20.25	55.16

Table 5: MSP.B (seed-weighted aggregation) results

aggregated signatures	**8**	**16**	**32**	**64**	**128**	**256**
Proving time (s)	35.73	48.48	65.37	103.75	184.02	327.30
Proof size (bytes)	5856	7936	10944	17312	30240	55744
Verification time (ms)	10.30	14.68	15.05	14.54	24.51	69.35

Table 6: Lottery index check results (range & uniqueness)

indices	**128**	**256**	**512**	**1024**	**2048**
Proving time (s)	4.08	6.77	13.41	37.39	122.49
Proof size (bytes)	960	1920	4576	14944	14944
Verification time (ms)	5.05	6.15	9.42	15.10	13.08

Table 7: Eligibility / stake-threshold check results

checks	**128**	**256**	**512**	**1024**	**2048**
Proving time (s)	10.14	12.78	17.36	26.39	44.56
Proof size (bytes)	1344	1920	2848	4800	8832
Verification time (ms)	6.77	6.39	7.10	11.24	12.59

3 Snarkifying Mithril(Theoretical)

3.1 SNARK

A SNARK for a relation $\mathcal{R}$ is a triple of PPT algorithms $\Pi = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$ defined as follows:

- $\mathsf{srs} \leftarrow \mathsf{Setup}(1^\lambda)$:On input security parameter λ , it outputs a structured reference string srs.
- $\pi \leftarrow \mathsf{Prove}(\mathsf{srs}, x, w)$: On input srs, a statement x and a witness w such that $(x, w) \in \mathcal{R}$, output a proof π.
- $0/1 \leftarrow \mathsf{Verify}(\mathsf{srs}, x, \pi)$:On input srs, a statement x, and a proof π, it outputs either 1 indicating accepting the statement or 0 for rejecting it.

3.2 BLS

$\Pi_{BLS} = (Setup, Sign, Ver)$

- $(sk, vk = g_1^{sk}) \leftarrow Setup()$
- $\sigma \leftarrow Sign(sk, msg)$, where $\sigma = H(msg)^{sk}$
- $1/0 \leftarrow Ver(vk, \sigma, msg)$, check $e(g_1, \sigma) = e(\mathsf{vk}, H(msg))$

3.3 Merkle tree

$\Pi_{MT} = (Create, Check)$

- $avk \leftarrow Create(\mathbf{VK})$
- $1/0 \leftarrow Check(vk, avk, path)$

3.4 SNARK-based Mithril

$\Pi_{\mathsf{F-Mithril}} = (\mathsf{Register}, \mathsf{EligibilityCheck}, \mathsf{CreateSig}, \mathsf{Verify}, \mathsf{Aggregate}, \mathsf{VerifyAggregate})$

$\Pi_{\mathsf{SNARK-Mithril}} = (\mathsf{PGen}, \mathsf{KGen}, \mathsf{Avk}, \mathsf{AvkProve}, \mathsf{AvkCheck}, \mathsf{Sig}, \mathsf{Ver}, \mathsf{ASig}, \mathsf{AVer})$

Mithril.PGen()

$$srs \leftarrow \mathrm{Setup}(1^\lambda)$$
$$\text{return } srs$$

Mithril.KGen(pp)

$$sk \leftarrow Z_q$$
$$vk \leftarrow g_1^{sk}$$
$$\text{return } (sk, vk)$$

Mithril.Avk(VK, w)

$$\mathbf{WVK} \leftarrow \{(w_i, vk_i) : i \in [n]\}$$
$$w \leftarrow \sum_{i \in [n]} w_i$$
$$avk \leftarrow MT.\mathrm{Create}(\mathbf{WVK})$$
$$\text{return } (avk, w)$$

Mithril.AvkProve(VK, w, avk, path srs)

$$\pi_1 \leftarrow \mathsf{SNARK.Prove}(srs, x = avk, w = (\mathbf{VK}, \mathbf{w}, \mathbf{path}))$$
$$(\mathcal{R} = \{(x = avk, w = (\mathbf{VK}, \mathbf{w}, \mathbf{path})) \quad | \quad \forall i \in [m],\ MT.\mathrm{Check}(avk, (w_i, vk_i), path_i) = 1\})$$
$$\text{return } \pi_1$$

Mithril.AvkCheck(avk, srs)

$$\text{return } \mathsf{SNARK.Verify}(srs, x = avk, \pi_1)$$

Mithril.Sig(sk, j, msg, w, $\mathbf{w}_i$)

$$\sigma \leftarrow \mathrm{BLS.Sign}(sk, msg)$$
$$\text{return } \sigma$$

Mithril.Ver(vk, j, msg, σ, w, w_i)

$$\text{return } \mathrm{BLS.Ver}(vk, \sigma, msg)$$

Mithril.ASig(S=$\{(\sigma_j, j) | j \in [m]\}$**)**

$$\boldsymbol{\sigma} \leftarrow [\{\sigma_j | j \in [m]\}]$$
$$e_\sigma \leftarrow H_p(\boldsymbol{\sigma})$$
$$e_j \leftarrow H_\lambda(j, e_\sigma)$$
$$\mu \leftarrow \prod_j \sigma_j^{e_j}$$
$$\pi_2 \leftarrow SNARK.Prove(srs, x = (\mu, e_\sigma), w = (\boldsymbol{\sigma}))$$
$$(R = \{x = (\mu, e_\sigma, , w), w = (\boldsymbol{\sigma}) \quad |$$
$$e_\sigma = H_p(\boldsymbol{\sigma}) \wedge e_j = H_\lambda(j, e_\sigma) \wedge \mu = \prod_j \sigma_j^{e_j}\})$$
$$\sigma^* \leftarrow (\mu, e_\sigma, \pi_2)$$
return σ^*

Mithril.AVer(ivk, msg, $\sigma*, w$)

$$r_1 \leftarrow \mathsf{SNARK.Verify}(srs, x = (\mu, e_\sigma), \pi_2)$$
$$r_2 \leftarrow \mathsf{BLS.Ver}(ivk, \mu, msg)$$
return $r_1 \wedge r_2$

(Here, ivk $= \prod_j vk_j^{e_j}$)

3.5 Remained Questions

1. It is unclear how to incorporate weights into BLS signatures, especially when performing aggregation in BLS multisignatures.

   In the case where all signers agree on the same message m, the aggregated signature can be written as

   $$\sigma_{agg} = \prod_i \sigma_i = H(m)^{\sum_i sk_i},$$

   and thus the corresponding aggregated public key is simply

   $$vk_{agg} = \prod_i vk_i.$$

   Verification then reduces to a single pairing equation

   $$e(g, \sigma_{agg}) \stackrel{?}{=} e(vk_{agg}, H(m)),$$

   which is compact and efficient.

   However, when each signer signs a *different* message m_i, the aggregated signature becomes

   $$\sigma_{agg} = \prod_i H(m_i)^{sk_i},$$

for which there does not exist a single aggregated public key. Instead, the verification requires

$$e(g, \sigma_{agg}) \stackrel{?}{=} \prod_i e(vk_i, H(m_i)),$$

where every individual public key vk_i and message m_i must be explicitly included in the verification. This implies that, unlike the single-message case, the verification cost is not succinct, and the aggregation does not yield a compact verification procedure.

2. The smart contract needs to verify both the Mithril.AvkCheck and Mithril.AVer proofs, i.e., it must verify two SNARK proofs and one BLS signature.

-

Mithril Bridge

```
Setup_{i,j}():
    return Mithril.PGen()

KGen_{i,j}(pp_{i,j}):
    return Mithril.KGen(pp_{i,j})

RoT_{i,j}(pp_{i,j}, vks_{i,j},C_i,C_j):
    [vk_P : P ∈ P_{i,j}] ← parse(vks_{i,j})
    get w from C_i
    avk ← Mithril.Avk(pp_{i,j}, [vk_P : P ∈ P_{i,j}])
    nonce ← Beacon(C_i) // latest round of C_i
    put nonce in bst
    rot_{i,j} ← (avk, w)
    aux_{i,j} ← (rot_{i,j}, [vk_P : P ∈ P_{i,j}])
    return (rot_{i,j}, aux_{i,j})

Br_{i,j}(r, m; C_i, C_j; bst) when r = 0:
    get rot_{i,j} from bst // P_{i,j} known
    code ← MITHBridge[rot_{i,j}].code
    cID ← Hash(code)
    btx_j ← (cID, install, (code, mem := ε))
    return (⊥, btx_j,⊥; bst)

Br_{i,j}(r, m; C_i, C_j; bst) when r ∈ [1, e −1]:
    return (⊥, ⊥,⊥; bst) // i.e., wait

Br_{i,j}(r, m; C_i, C_j; bst) when r = ke − Δ − 1, k ∈ ℕ:
    get (P, sk) from bst
    get pp_{i,j} from bst
    (sk', vk') ← Mithril.KGen(pp_{i,j})
    put sk in bst
    σ' ← DS.Sig(sk, vk')
    return (⊥, ⊥ , (P, vk', σ'); bst)

Br_{i,j}(r, m; C_i, C_j; bst) when r ∈ [ke − Δ, ke − 1], k ∈ ℕ:
    put m in bst
    return (⊥, ⊥, ⊥; bst)

Br_{i,j}(r, m; C_i, C_j; bst) when r = k e, k ∈ ℕ:
    get (P, sk) from bst
    get [vk'_P : P ∈ P_{i,j}] from bst  // in collected m
    VK' ← [vk'_P : P ∈ P_{i,j} ∧ DS.Ver(vk_P, vk'_P) = 1]
    avk' ← Mithril.Avk(pp_{i,j}, VK')
```

```
Br_{i,j}(r, m; C_i, C_j; bst) when r = k e + Δ, k ∈ ℕ:
// aggregator
    put m in bst
    get msg from bst
    get [vk_P: P ∈ P_{i,j}] from bst
    get w from C_i for round r = k e
    get VK from bst
    form WVK from w and VK
    compute w using w
    get w_P from w
    get T := [(vk, j*, σ)] from bst
    S ← [(vk, j*, σ) ∈ T : Mithril.Ver(vk, j*, msg, σ, w_i, w)]
    σ* ← Mithril.ASig(VK,w, msg, S)
    return (⊥, (cID, (msg, σ*)), ⊥; bst)

Br_{i,j}(r, m; C_i, C_j; bst) when r ∈ [k e + Δ + 1, (k+1) e − 1], k ∈ ℕ:
    return (⊥, ⊥, ⊥; bst) // do nothing

GetVerData_{i,j}(genesis_j,rot_{i,j}, L_j):
    LS ← LedgerState_j[genesis_j](L_j)
    cID ← Hash(MITHBridge[rot_{i,j}].code)
    (_ , mem, _) ← LS[cID]
    return mem

R*_{i,j}((cpl, x, h), C):
    return (Header_i(C, cpl) = h ∧ R_{i,j}(x, L_i(C) up to cpl))

Certify_{i,j}(rot_{i,j}, x, C_i): // identical to the others
    get crs_{i,j} from rot_{i,j}
    cpl ← Floor(r/Δ_i)
    h ← Header_i(C, cpl)
    π ← Snark.Prove(crs_{i,j}, (cpl, x, h), C)
    return (cpl, π)

Verify_{i,j}(mem, x, π):
    get crs_{i,j} from mem
    get list of h from mem for the cpl
    return Snark.Verify(crs_{i,j}, (cpl, x, h), π)

Contract MITHBridge[rot_{i,j}]:

update(msg, b):
    get latest avk from mem
    if avk = ⊥: get avk from rot_{i,j}
    (cpl, h, w, avk, nonce) ← msg.body
    get w from w
```

```
get w' from C_i at the end of round r = k e
cpl ← floor(r/Δ_i) // Δ_i = liveness delay
h ← Hash(L_i(C_i up to cpl))
nonce' ← Beacon(C_i round r)
get nonce from bst
msg.topic ← r            // = k e
msg.body ← (cpl, h, w', avk', nonce')
put msg in bst  // includes nonce'
get w from C_i for round (k - 1)*e
compute w using w
get w_i from w
m ← [ ]
for j* ∈ [m]:
    41.1.   σ ← Mithril.Sig(sk, j*, msg, w_i, w)
    41.2.   if σ ≠ ⊥ : m ← m : (vk_P, j*, σ)
if m = [] : m ← ⊥
return (⊥, ⊥, m; bst)

Br_{i,j}(r, m; C_i, C_j; bst) when r ∈ [k e + 1, k e + Δ - 1], k ∈ ℕ:
put m in bst
return (⊥, ⊥, ⊥; bst)
```

```
check Mithril.AVer(avk, msg, σ, w)
mem ← mem : (cpl, h, w, avk, nonce) // keep old h's and avk

verify(x, π):
if Verify_{i,j}(mem, x, π): put (ok, x, π) in mem
```